

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы данных»
Тема: CRM для кадровиков

Студенты гр. 3341

Бойцов В. А.

Романов А. К.

Максимова Е. Д.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

НА ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ

Студенты

Бойцов В. А.

Романов А. К.

Максимова Е. Д.

Группа 3341

Тема проекта: Разработка CRM для кадровиков

Исходные данные: необходимо разработать веб-приложение, реализующее функционал CRM для отдела кадров. В качестве базы данных использовать neo4j.

Содержание пояснительной записки:

«Введение»

«Сценарии использования»

«Модель данных»

«Разработанное приложение»

«Выводы»

«Список источников»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания: 10.02.2026

Дата сдачи реферата: 21.05.2026

Дата защиты реферата: 21.05.2026

Студенты гр. 3341

Бойцов В. А.

Романов А. К.

Максимова Е. Д.

Преподаватель

Заславский М.М.

АННОТАЦИЯ

В данной работе было разработано web-приложение, реализующее функциональность системы CRM для кадровиков. Было выполнено написание сценариев использования и создания макета, затем по итерациям разработано приложение. В основе приложения использовалась СУБД neo4j, FastAPI для бэкенд части и React для фронтенд части.

SUMMARY

This project involved developing a web application implementing the functionality of a CRM system for HR professionals. Use cases were written and a mockup was created, followed by iterative development of the application. The application was based on the neo4j DBMS, FastAPI for the backend, and React for the frontend.

СОДЕРЖАНИЕ

Введение	6
1. Сценарии использования	7
1.1. Макет UI	7
1.2. Use Case	8
2. Модель данных	24
2.1. Нереляционная модель	24
2.2. Реляционная модель	36
2.3. Сравнение моделей	55
3. Разработанное приложение	58
3.1. Краткое описание	58
3.2. Используемые технологии	59
3.3. Скриншоты	60
Заключение	63
Список литературы	65
Приложение А: документация по сборке и развёртыванию приложения	66
Приложение Б: инструкция для пользователя	67

ВВЕДЕНИЕ

С растущим оборотом найма сотрудников в современных компаниях возрастает потребность в автоматизированных системах, упрощающих анализ кандидатов, создание вакансий и процедуру найма в целом. Данную проблему решают CRM: позволяют упрощать и систематизировать процессы с помощью автоматизации, реализованной в составе ПО.

Таким образом, требуется разработать систему, автоматизирующую:

- создание вакансий и отслеживание их статусов;
- отслеживание кандидатов на определенные позиции и их статусы;
- отслеживание разнообразных процессов и этапов найма: тестовые задания, собеседования, офферы, и управление ими.

Предлагаемое решение – web-приложение, состоящее из трёх компонентов. Хранение данных реализуется с помощью графовой базы данных neo4j: использование графов хорошо отражает связи между сущностями. Бэкенд реализуется с помощью FastAPI и заточен под внедрение автоматизации и разделение ответственности пользователей с разными ролями (HR, технический специалист и так далее). Фронтенд реализуется с помощью React и предоставляет реактивный интерфейс для удобного управления процессами, фильтрации сущностей и просмотра статистики.

Основные требования к решению формулируются следующим образом:

1. Автоматизация основных процессов найма сотрудников;
2. Возможность отслеживания, фильтрации и просмотра всех сущностей: кандидатов, тестовых заданий, вакансий и т.д.;
3. Возможность импорта и экспорта данных приложения в человекочитаемом формате (был выбран JSON);
4. Разделение ответственности между пользователями: HR отвечает за подбор кандидатов и их сопровождение в процессе отбора, технический специалист – оценку профессиональных навыков кандидата, менеджер (руководитель) – согласование и подписание оффера.

1. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

1.1. Макет UI

В начале разработки приложения был разработан макет фронтенда приложения, показанный на рис. 1.

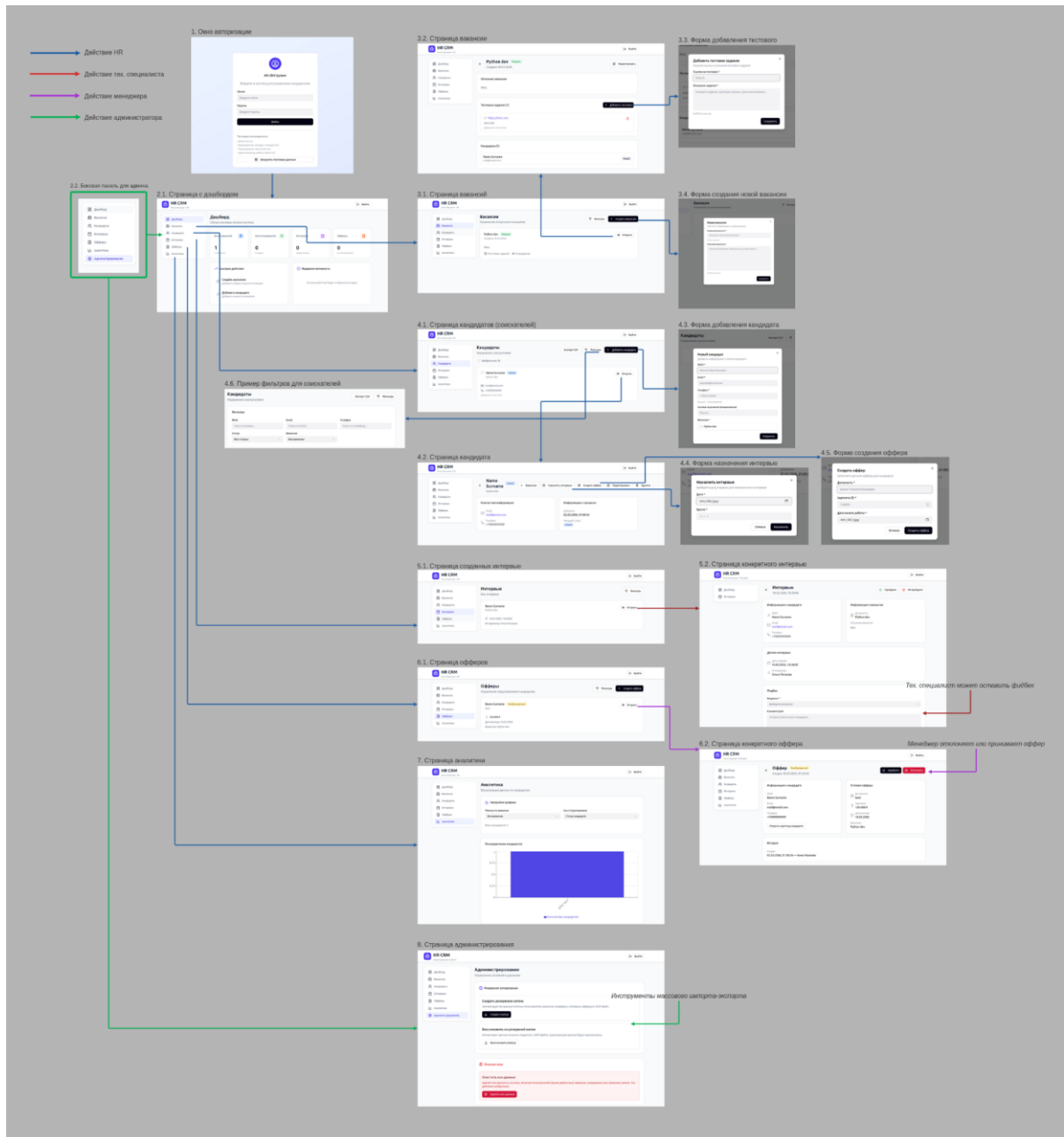


Рисунок 1 – разработанный макет приложения

1.2. Use case

Ниже приведены разработанные сценарии использования приложения.

UC-00 Регистрация пользователя

Действующее лицо: Пользователь (HR / Руководитель / Технический специалист)

Предусловие:

- Пользователь не авторизован.
- Пользователь находится на странице «Регистрация».
- В системе отсутствует учетная запись с указанным email.

Основной сценарий:

1. Пользователь открывает страницу «Регистрация».
2. Вводит ФИО (строка от 3 до 150 символов).
3. Вводит email (формат user@example.com).
4. Вводит пароль (не менее 6 символов).
5. Вводит подтверждение пароля.
6. Выбирает роль из выпадающего списка: HR, Руководитель, Технический специалист
7. Нажимает кнопку «Зарегистрироваться».
8. Система выполняет валидацию: корректность формата email, совпадение паролей, минимальную длину пароля, заполненность обязательных полей.
9. Система проверяет уникальность email.
10. Система создает учетную запись.
11. Система присваивает выбранную роль.
12. Система перенаправляет пользователя на страницу авторизации.

Альтернативный сценарий:

- 9а. Система обнаруживает существующий email.
10а. Отображается сообщение «Пользователь с таким email уже зарегистрирован».
- 11а. Регистрация не выполняется.
- 8а. Поля «Пароль» и «Подтверждение пароля» различаются.
9а. Система отображает «Пароли не совпадают».
- 10а. Пользователь остается на странице регистрации.
- Некорректные данные. Система отображает соответствующее сообщение об ошибке.
- Потеря соединения с сервером при создании учетной записи.
Система отображает «Ошибка соединения. Повторите попытку позже».
- Пользователь нажимает «Отмена» или закрывает страницу до завершения регистрации. Данные не сохраняются.

Результат: Учетная запись создана.

УС-01 Авторизация пользователя

Действующее лицо: Пользователь (HR / Руководитель / Техспец / Администратор)

Предусловие:

- Пользователь зарегистрирован в системе.
- Пользователь находится на странице входа.

Основной сценарий:

1. Пользователь вводит email в текстовое поле формата user@example.com.

2. Пользователь вводит пароль длиной не менее 6 символов.
3. Пользователь нажимает кнопку «Войти».
4. Система проверяет корректность учетных данных.
5. Система определяет роль пользователя.
6. Система перенаправляет пользователя на главную страницу согласно роли.

Альтернативный сценарий:

- 4а. Система определяет, что email или пароль введены неверно. Система отображает сообщение «Неверный email или пароль».
- 4б. Во время проверки учетных данных возникает ошибка соединения с сервером. Система отображает сообщение «Ошибка соединения. Повторите попытку».

Пользователь закрывает страницу входа.

Результат: Пользователь авторизован и получает доступ к функциям согласно роли.

UC-02 Создание вакансии

Действующее лицо: HR

Предусловие:

- HR авторизован.
- Открыт раздел «Вакансии».

Основной сценарий:

1. HR нажимает кнопку «Создать вакансию».
2. HR вводит название вакансии (строка до 100 символов).

3. HR вводит описание вакансии (текст до 5000 символов).
4. HR выбирает статус «Открыта».
5. HR нажимает «Сохранить».
6. Система создает запись вакансии.
7. Система отображает карточку вакансии.

Альтернативный сценарий: незаполненные обязательные поля

- 3а. HR оставляет поле «Название» пустым.
- 4а. Система отображает сообщение «Поле обязательно для заполнения».
- 6а. Ошибка записи в базу данных.
- 7а. Система отображает сообщение «Не удалось создать вакансию».
- HR нажимает «Отмена» до сохранения.
 - HR выходит закрывает страницу с вакансиями до нажатия кнопки “Сохранить”

Результат: Вакансия создана и доступна в списке вакансий.

УС-03 Ручное добавление кандидата

Действующее лицо: HR

Предусловие:

- HR авторизован.
- В системе существует хотя бы одна вакансия.

Основной сценарий:

1. HR нажимает «Добавить кандидата».
2. HR вводит ФИО (строка до 150 символов).
3. HR вводит email (валидный формат).
4. HR вводит номер телефона (формат +7XXXXXXXXXX).

5. (Опционально) HR прикрепляет ссылку на резюме кандидата
6. HR выбирает вакансию из выпадающего списка.
7. HR нажимает «Сохранить».
8. Система создает карточку кандидата.
9. Система устанавливает статус «Новый».

Альтернативный сценарий: некорректный email

- 3а. HR вводит email без символа «@».
- 4а. Система отображает сообщение «Некорректный формат email».
- 4б. HR вводит не соответствующий формату номер
- 5б. Система отображает сообщение «Некорректный номер телефона».
- 7а. Ошибка соединения с сервером. Система отображает сообщение «Ошибка сохранения данных».
 - HR нажимает «Отмена».
 - HR выходит закрывает страницу с кандидатами до нажатия кнопки “Сохранить”

Результат: Кандидат создан и связан с выбранной вакансией.

УС-04 Редактирование информации о кандидате

Действующее лицо: HR

Предусловие:

- HR авторизован.
- Карточка кандидата существует.

Основной сценарий:

1. HR открывает раздел «Кандидаты».
2. HR вводит ФИО кандидата в поле поиска (строка до 150 символов).

3. Система отображает результаты поиска.
4. HR открывает карточку кандидата.
5. HR нажимает «Редактировать».
6. HR изменяет одно или несколько полей: ФИО (строка); email (валидный формат); телефон (+7XXXXXXXXXX).
7. HR нажимает «Сохранить».
8. Система валидирует данные.
9. Система сохраняет изменения.
10. Система отображает обновлённую карточку.

Альтернативный сценарий:

- 8а. Email введён без символа «@».
- 9а. Система отображает «Некорректный формат email».
- 9б. Ошибка соединения с БД.
- 10б. Система отображает сообщение об ошибке.
- HR нажимает «Отмена» до сохранения.
 - HR закрывает страницу до сохранения.

Результат: Информация о кандидате обновлена.

UC-05 Массовый импорт кандидатов (CSV)

Действующее лицо: HR / Администратор

Предусловие:

- Пользователь авторизован.
- Подготовлен CSV-файл с колонками: *full_name*, *email*, *phone*, *vacancy*, *job_title*, *skills*.

Основной сценарий:

1. Пользователь открывает раздел «Импорт».
2. Нажимает «Загрузить CSV».
3. Выбирает файл формата .csv.
4. Система проверяет структуру колонок.
5. Система отображает предпросмотр первых 10 строк.
6. Пользователь нажимает «Подтвердить импорт».
7. Система создает кандидатов.
8. Система отображает отчет: «Импортировано 25 записей».

Альтернативный сценарий:

- 4а. Система не находит колонку название колонки.
- 5а. Отображается сообщение «Отсутствует обязательное поле название колонки».
- 7а. Во время импорта происходит потеря соединения.
 - Система сообщает “Не удалось импортировать данные. Проверьте соединение”
- 5а. Пользователь нажимает «Отмена».

Результат: Все кандидаты из файла добавлены в систему.

УС-06 Прикрепление тестового к вакансии

Действующее лицо: HR

Предусловие:

- Вакансия существует.

Основной сценарий:

1. HR открывает карточку вакансии.
2. HR нажимает «Добавить тестовое».
3. HR вставляет ссылку (строка URL формата https://...).
4. HR вводит описание задания (до 2000 символов).
5. HR нажимает «Сохранить».
6. Система сохраняет тестовое и связывает его с вакансией.

Альтернативный сценарий: неверный URL

- 4а. Ссылка не начинается с http/https.
- 5а. Система отображает «Некорректный формат ссылки».
- Ошибка записи в БД.
 - HR закрывает форму.

Результат: Тестовое прикреплено к вакансии.

UC-07 Отправка тестового кандидату

Действующее лицо: HR

Предусловие:

- Кандидат существует.
- Кандидат связан с вакансией.
- У вакансии есть тестовое.

Основной сценарий:

1. HR открывает карточку кандидата.
2. HR нажимает «Отправить тестовое».

3. Система автоматически подставляет: ФИО кандидата; название вакансии; ссылку на тестовое.

4. HR нажимает «Отправить».

5. Система фиксирует факт отправки.

6. Статус кандидата меняется на «Test Assigned».

Альтернативный сценарий: отсутствует email.

- 3а. У кандидата отсутствует email.

4а. Система отображает «Email не указан».

- Ошибка отправки письма.

- HR закрывает окно отправки.

Результат: Кандидату отправлено тестовое.

УС-08 Назначение интервью

Действующее лицо: HR

Предусловие:

- Кандидат существует.
- В системе есть зарегистрированный техспециалист.

Основной сценарий:

1. HR заходит в раздел “Кандидаты”
2. HR открывает карточку кандидата.
3. Нажимает «Назначить интервью».
4. HR выбирает дату в поле ввода с датой DatePicker
5. HR выбирает дату в поле ввода с датой TimePicker
6. HR выбирает техспециалиста.

7. Нажимает «Назначить».
8. Система создает запись интервью.
9. Кандидат получает статус «Tech Interview».

Альтернативный сценарий:

- 4-5а. Выбранное дата и время уже занято.
- 5а. Система отображает «Выберите другое время».
- 7а. Ошибка сохранения информации об интервью.
- 8а. Система отображает сообщение об ошибке.
- HR закрывает форму назначения.

Результат: Интервью назначено и связано с кандидатом и тех. специалистом.

УС-09 Проведение интервью и оставление фидбека

Действующее лицо: Технический специалист

Предусловие:

- Интервью назначено.
- Кандидат имеет статус «Tech Interview».
- Технический специалист авторизован в системе.

Основной сценарий:

1. Технический специалист открывает раздел «Мои интервью».
2. Выбирает интервью из списка по ФИО кандидата или дате.
3. Система отображает: ФИО кандидата; ссылку или PDF-резюме; описание вакансии; дату и время интервью.
4. Технический специалист проводит интервью (вне системы).
5. После завершения интервью нажимает «Оставить фидбек».

6. Вводит комментарий (текст до 2000 символов).
7. Выбирает результат: «Пройдено» или «Не пройдено».
8. Нажимает «Сохранить».
9. Система: сохраняет комментарий, фиксирует ID специалиста и дату, обновляет статус кандидата: «Interview Passed» или «Interview Failed».

Альтернативный сценарий: не выбран статус

- 3а. Резюме отсутствует. Система отображает «Резюме не прикреплено». 4а. Интервью может быть проведено, но информация неполная.
- 7а. Пользователь не выбирает результат. 8а. Система отображает сообщение «Выберите решение». 9а. Данные не сохраняются.
- Потеря соединения с сервером при сохранении. Система отображает сообщение: «Изменения не сохранены. Проверьте подключение к сети».
- Статус кандидата остаётся без изменений.
- Технический специалист закрывает страницу без нажатия «Сохранить». Фидбек не фиксируется.

Результат:

- Решение по интервью сохранено.
- Статус кандидата обновлён.
- HR получает доступ к результату интервью.

УС-10 Отправка оффера

Действующее лицо: HR

Предусловие:

- Кандидат имеет статус «Approved».

Основной сценарий:

1. HR открывает карточку кандидата.
2. HR нажимает «Сформировать оффер».
3. HR вводит: должность; размер зарплаты (число); дата выхода (дата).
4. HR нажимает «Отправить на согласование».
5. Система направляет оффер руководителю.

Альтернативный сценарий:

- Ошибка сохранения документа.
- HR закрывает форму.

Результат: Оффер отправлен руководителю.

УС-11 Подписание оффера

Действующее лицо: Руководитель

Предусловие:

- HR направил оффер на согласование.

Основной сценарий:

1. Руководитель открывает раздел «Офферы на подпись».
2. Открывает документ.
3. Проверяет условия.
4. Нажимает «Подписать».
5. Система фиксирует статус «Подписан».

Альтернативный сценарий:

- Руководитель нажимает «Отклонить».
- Ошибка сохранения решения.

Результат: Оффер подписан или отклонён.

UC-12 Просмотр аналитики

Действующее лицо: Руководитель / HR

Предусловие:

- В системе есть данные.

Основной сценарий:

1. Пользователь открывает раздел «Аналитика».
2. Выбирает фильтр (например, вакансии = Python Developer).
3. Выбирает ось X (Статус).
4. Выбирает ось Y (Количество).
5. Нажимает «Построить».
6. Система выполняет группировку данных.
7. Отображается столбчатая диаграмма.

Альтернативный сценарий:

- 6a. Данные отсутствуют. Система отображает «Данные отсутствуют».
- 6b. Ошибка запроса к БД.
- Пользователь закрывает страницу с аналитикой.

Результат: Пользователь получает визуализацию данных.

УС-13 Полный экспорт системы (Backup)

Действующее лицо: Администратор

Предусловие:

- Администратор авторизован.

Основной сценарий:

1. Администратор открывает «Администрирование».
2. Нажимает «Создать резервную копию».
3. Система формирует JSON-файл со всеми узлами и связями.
4. Начинается скачивание файла.

Альтернативный сценарий:

Нежелательное завершение.

- Ошибка соединения с базой данных.
- Отмена
- Администратор нажимает «Отмена».

Результат: JSON-файл сохранен.

УС-14 Полный импорт системы (восстановление из Backup)

Действующее лицо: Администратор

Предусловие:

- Администратор авторизован в системе.
- У администратора есть файл резервной копии формата JSON.
- Размер файла не превышает установленного лимита (например, 50 МБ).

Основной сценарий:

1. Администратор открывает раздел «Администрирование».
2. Нажимает кнопку «Восстановить из резервной копии».
3. Система отображает предупреждение:
«Текущие данные будут заменены. Продолжить?»
4. Администратор нажимает «Продолжить».
5. Администратор выбирает файл формата json.
6. Нажимает «Загрузить».
7. Система: проверяет формат файла, проверяет целостность структуры (наличие обязательных узлов: users, vacancies, candidates, interviews, offers), проверяет корректность связей между сущностями.
8. Система очищает текущие данные.
9. Система импортирует данные из файла.
10. Система отображает сообщение «Восстановление завершено успешно».

Альтернативный сценарий:

- 7а. Система обнаруживает, что файл не JSON. Отображается сообщение «Допустим только формат JSON».
 - 7б. Отсутствует обязательный раздел (например, users).
- 8б. Система отображает: «Файл резервной копии поврежден или имеет неверную структуру».
- Потеря соединения с базой данных во время импорта. Система отображает сообщение «Ошибка восстановления. Данные могли быть изменены частично».

- Система фиксирует событие в журнале ошибок.

Результат: Система полностью восстановлена из резервной копии

2. МОДЕЛЬ ДАННЫХ

2.1. Нереляционная модель

Графическое представление модели

Структуру графовой базы данных демонстрирует ER-диаграмма, показанная на рис. 2.

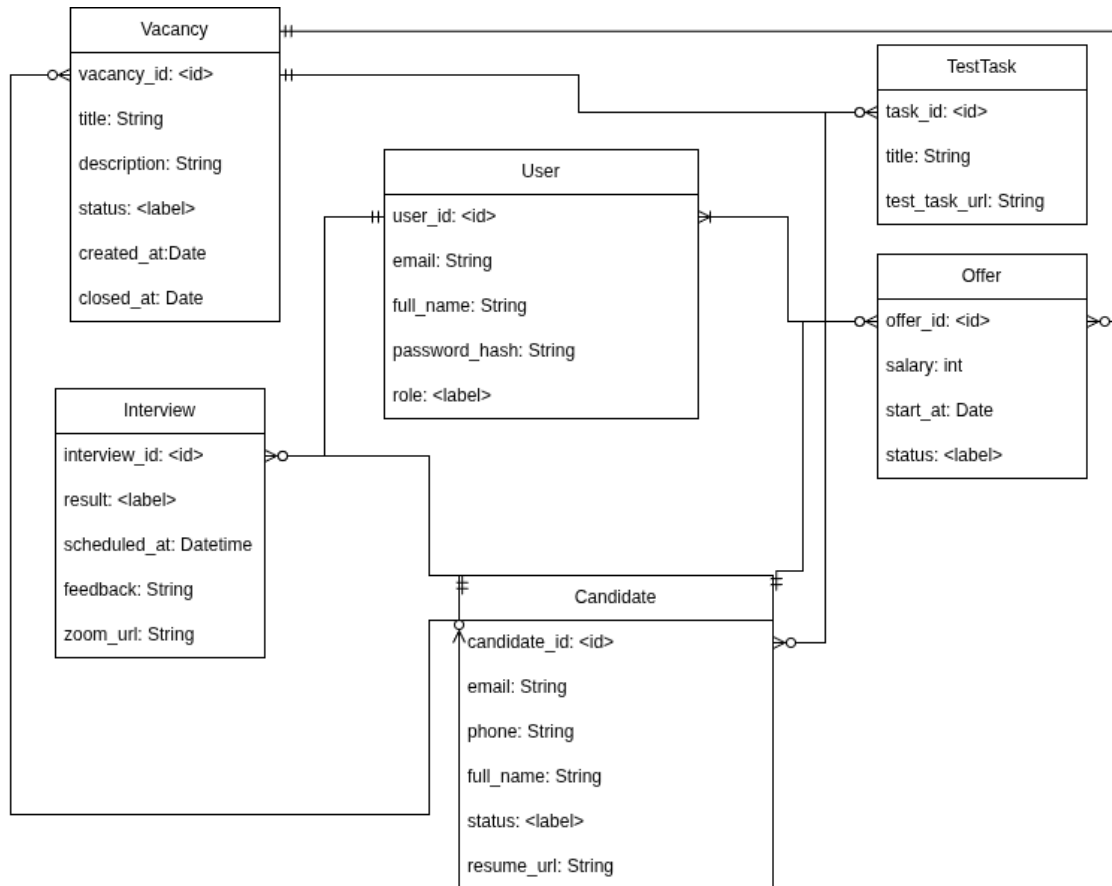


Рисунок 2 – ER-диаграмма нереляционной модели

Структура БД демонстрируется на примере заполнения БД, показанного на рис. 3.

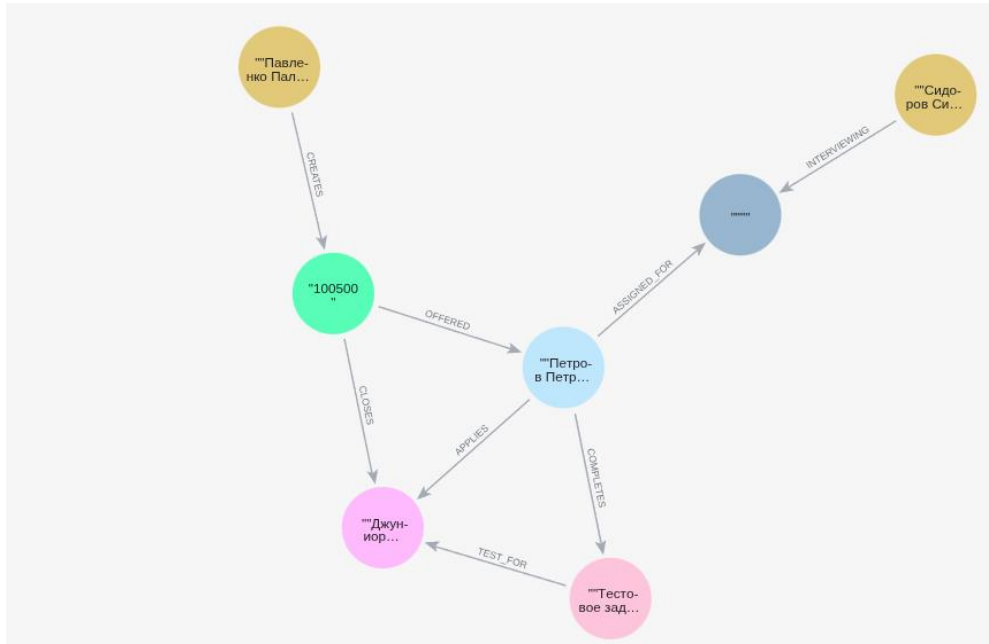


Рисунок 3 – пример заполнения БД

Описание

Используется графовая БД neo4j, сущности являются узлами графа, рёбра в графе определяют связи между сущностями.

Сущности

- 1. Vacansy — Вакансия

В качестве идентификатора используем встроенный id узла, его вес включён в 15 байт узла. Атрибуты сущности перечислены в табл. 1.

Таблица 1: атрибуты сущности Vacansy

Атрибут	Назначение	Тип	Вес (байт)
vacansy_id	id	< id >	- (скрытые затраты)
title	Название позиции	String	32 + 30 = 62

description	Описание вакансии	String	$32 + 200 = 232$
created_at	Дата создания	Date (хранится как long)	$32 + 8 = 40$
closed_at	Дата закрытия	Date	$32 + 8 = 40$
status	Статус вакансии	Label	$32 + 6 = 38$

Значения status: OPEN, CLOSED

Скрытые затраты: запись узла (nodestore): 15 байт (включает внутренний id, ссылки)

Итого на узел: $15 + 62 + 232 + 42 + 40 + 40 + 32 = 469$ байт

2. TestTask — Тестовое задание

Тестовое задание, прикрепляется к вакансии. Атрибуты TestTask показаны в табл. 2.

Таблица 2 – атрибуты сущности TestTask

Атрибут	Назначение	Тип	Вес (байт)
task_id	id	< id >	- (скрытые затраты)
title	Название задания	String	$32 + 30 = 62$
test_task_url	Условия задания	String	$32 + 100 = 132$

Скрытые затраты: запись узла: 15 байт

Итого на узел: $15 + 62 + 132 = 209$ байт

3. User — Сотрудник системы

Пользователь CRM - HR, Технический специалист, руководитель или админ. Права доступа зависят от роли пользователя. Атрибуты User показаны в табл. 3.

Таблица 3 – атрибуты сущности User

Атрибут	Назначение	Тип	Вес (байт)
user_id	id	< id >	- (скрытые затраты)
email	Логин	String	32 + 30 = 62
full_name	ФИО	String	32 + 40 = 72
password_hash	Хеш пароля	String	32 + 64 = 96
role	Роль	Label	32 + 8 = 40

Значения role: ADMIN, HR, TECH_SPEC, MANAGER

Скрытые затраты: запись узла: 15 байт

Итого на узел: $15 + 62 + 72 + 96 + 40 = 285$ байт

4. Interview — Интервью

Информация о назначенных и прошедших собеседованиях - время, результат, отзыв. С каждым интервью связан кандидат и специалист, которые должны быть на собеседовании. Атрибуты сущности Interview показаны в табл. 4.

Таблица 4 – атрибуты сущности Interview

Атрибут	Назначение	Тип	Вес (байт)
interview_id	id	< id >	- (скрытые затраты)
scheduled_at	Дата+время	Timestamp (long)	32 + 8 = 40

feedback	Отзыв	String	$32 + 100 = 132$
zoom_url	Ссылка на встречу	String	$32 + 50 = 82$
result	Итог собеседования	Label	$32 + 16 = 48$

Значение result: AWAIT_INTERVIEW, INTERVIEW_PASSED, INTERVIEW_FAILED

Скрытые затраты: запись узла: 15 байт

Итого на узел: $15 + 40 + 40 + 132 + 82 + 48 = 317$ байт

5. Candidate — Соискатель

Хранит контактные кандидатов. Атрибуты сущности Candidate показаны в табл. 5.

Таблица 5 – атрибуты сущности Candidate

Атрибут	Назначение	Тип	Вес (байт)
candidate_id	id	< id >	- (скрытые затраты)
email	Почта	String	$32 + 30 = 62$
full_name	ФИО	String	$32 + 40 = 72$
phone	Телефон	String	$32 + 20 = 52$
resume_url	Ссылка на резюме	String	$32 + 100 = 132$
status	Статус кандидата	Label	$32 + 8 = 40$

Значения status: NEW, TEST, INTERVIEW, OFFER, REJECTED, HIRED

Скрытые затраты: запись узла: 15 байт

Итого на узел: $15 + 62 + 72 + 52 + 132 + 40 = 373$ байт

6. Offer — Оффер

Документ с финальными условиями работы. Направляется на подпись руководителю внутри CRM и кандидату. Атрибуты сущности Offer показаны в табл. 6.

Таблица 6 – атрибуты сущности Offer

Атрибут	Назначение	Тип	Вес (байт)
offer_id	id	< id >	- (скрытые затраты)
salary	Зарплата	Integer	$32 + 8 = 40$
start_at	Дата выхода	Date	$32 + 8 = 40$
status	Статус оффера	Label	$32 + 12 = 44$

Значения status: PENDING, APPROVED_MNG, REJECTED_MNG, APPROVED_CND, REJECTED_CNF

Скрытые затраты: запись узла: 15 байт

Итого на узел: $15 + 40 + 40 + 44 = 139$ байт

Связи

Каждая связь занимает 34 байта.

Между сущностями реализованы следующие связи:

- APPLIES: связывает кандидата с вакансией, на которую он подаётся: Candidate → Vacancy.
- COMPLETES: связывает кандидата с тестовым, которое он выполняет: Candidate → TestTask.
- TEST_FOR: связывает тестовое задание с вакансией TestTask: → Vacancy.

- ASSIGNED_FOR: связывает кандидата с интервью: Candidate → Interview.
- INTERVIEWING: связывает пользователя (роль "Тех. специалист") с интервью: User → Interview.
- CREATES: связывает пользователя (роль "Руководитель") с оффером: User → Offer.
- OFFERED: связывает оффер с кандидатом: Offer → Candidate.
- CLOSES: связывает оффер с вакансией: Offer → Vacancy.

Итоговое резюме по весу узлов в СУБД Neo4j показано в табл. 7.

Таблица 7 – сводка по весу узлов в СУБД Neo4j

Сущность	Вес с типовыми длинами строк
TestTask	~209 байт
Vacancy	~469 байт
User	~285 байт
Interview	~317 байт
Candidate	~373 байта
Offer	~139 байт

Оценка объёма информации

Для сохранения в БД по одному экземпляру каждой сущности, согласно таблице выше, требуется ~1800 байт.

Зависимость объёма от числа кандидатов

Методология:

- Пусть N - число кандидатов, хранимое в базе.

- Пусть число сотрудников системы: $N/10$
- Пусть число вакансий равно числу кандидатов;
- Каждый кандидат подаёт в среднем 5 откликов;
- Каждому отклику соответствует одно собеседование;
- Каждой вакансии соответствует одно тестовое задание и один оффер.

Формула $S(N)$:

Тогда примерный объём данных S в зависимости от числа кандидатов N выражается формулой: $S(N) \sim N \times (373 + 469 + 5 \times 34 + 5 \times 317 + 209 + 139 + 16 \times 34) + N/10 = 3500 \times N + N/10$ байт

Избыточность данных

Посчитаем полезный объем каждой сущности. В него не входит вес узла, лейблов и оверхед на создание полей. (Вес лейбла заменим на вес enum-значения, достаточно 1 байта). Рассчитанная избыточность показана в табл. 8.

Таблица 8 – избыточность на сущность в СУБД Neo4j

Сущность	Избыточность
Vacancy	$469/247 = 1.90$
TestTask	$209/130 = 1.60$
User	$285/135 = 2.11$
Interview	$317/159 = 1.99$
Candidate	$373/191 = 1.95$
Offer	$139/17 = 8.17$

Средняя избыточность на узел: 2.95

Общая избыточность рассчитывается следующим образом:

$$\sum nosql = 469 + 209 + 285 + 317 + 373 + 139$$

$$\sum clean = 247 + 130 + 135 + 159 + 191 + 18$$

$$R_coeff = \sum nosql / \sum clean = 2.03$$

Таким образом, избыточность (Redundancy) равна $R \approx 2.03 \times V$, где V - объем полезных данных. Или $R \approx 2.95 \times N$, где N - количество узлов.

Направление роста

Между размером БД и количеством узлов прямая зависимость. Это применимо ко всем сущностям.

Добавление n экземпляров сущности увеличивает размер на $n * size$, где $size$ равен размеру узла этой сущности.

На вес также влияет количество отношений между сущностями (рёбер между узлами). Создание каждой связи будет увеличивать вес БД, вес одной связи - 34 байта.

- Создание новой сущности *Candidate* будет порождать связь с сущностью *Vacancy* (+34 байта).
- Создание тестового задания *TestTask* будет порождать связь с сущностью *Vacancy* (+34 байта).
- Выдача тестового кандидату будет порождать связь между *Candidate* и *TestTask* (+34 байта).
- Назначение интервью будет порождать создание двух связей (+68 байт): *Interview* и *Candidate*, *Interview* и *User*.
- Создание оффера будет порождать три связи (+112 байт): *User* и *Offer*, *Offer* и *Vacancy*, *Offer* и *Candidate*.

Таким образом, наибольший рост база будет испытывать при создании новых офферов и назначении интервью.

Примеры данных

Пример запросов для заполнения БД тестовыми данными.

Добавление пользователей:


```

CREATE (n:User:Manager{full_name: "Павленко Павел Павлович", email:
"manager@mail.com", password_hash: "manager_password_hash"})
CREATE (n:User:HR{full_name: "Валерьева Валерия Валерьевна", email:
"hr@mail.com", password_hash: "hr_password_hash"})
CREATE (n:User:Tech_spec{full_name: "Сидоров Сидор Сидорович",
email: "tech@mail.com", password_hash: "tech_password_hash"})

```

Добавление кандидата:

```

CREATE (n:Candidate:Offer{full_name: "Петров Пётр Петрович", email:
"candidate@mail.com", phone: "+78005553535", resume_url:
"https://resume.com/resume1"})

```

Добавление вакансии:

```

CREATE (n:Vacancy:Open{title: "Джуниор разработчик", description:
"Младший разработчик на старшую позицию", created_at: date()})
MATCH (v:Vacancy), (c:Candidate) CREATE (c)-[r:APPLIES]->(v)

```

Добавление тестового задания:

```

MATCH (c:Candidate), (v:Vacancy)
CREATE (t:TestTask{title: "Тестовое задание", test_task_url:
"https://test/test1"})
CREATE (c)-[:COMPLETES]->(t)
CREATE (t)-[:TEST_FOR]->(v)

```

Добавление интервью:

```

MATCH (c:Candidate), (s:Tech_spec)
CREATE (i:Interview:Interview_passed{feedback: "Интервью прошло
успешно", zoom_url: "https://zoom.com/sobes", scheduled_at: datetime()})
CREATE (c)-[:ASSIGNED_FOR]->(i)
CREATE (s)-[:INTERVIEWING]->(i)

```

Добавление оффера:

```

MATCH (c:Candidate), (v:Vacancy), (m:Manager)
CREATE (o:Offer:Pending{salary: 100500, start_at: date()})
CREATE (m)-[:CREATES]->(o)
CREATE (o)-[:CLOSES]->(v)
CREATE (c)-[:APPLIES]->(o)

```

Пример заполнения БД показан на рис. 4.

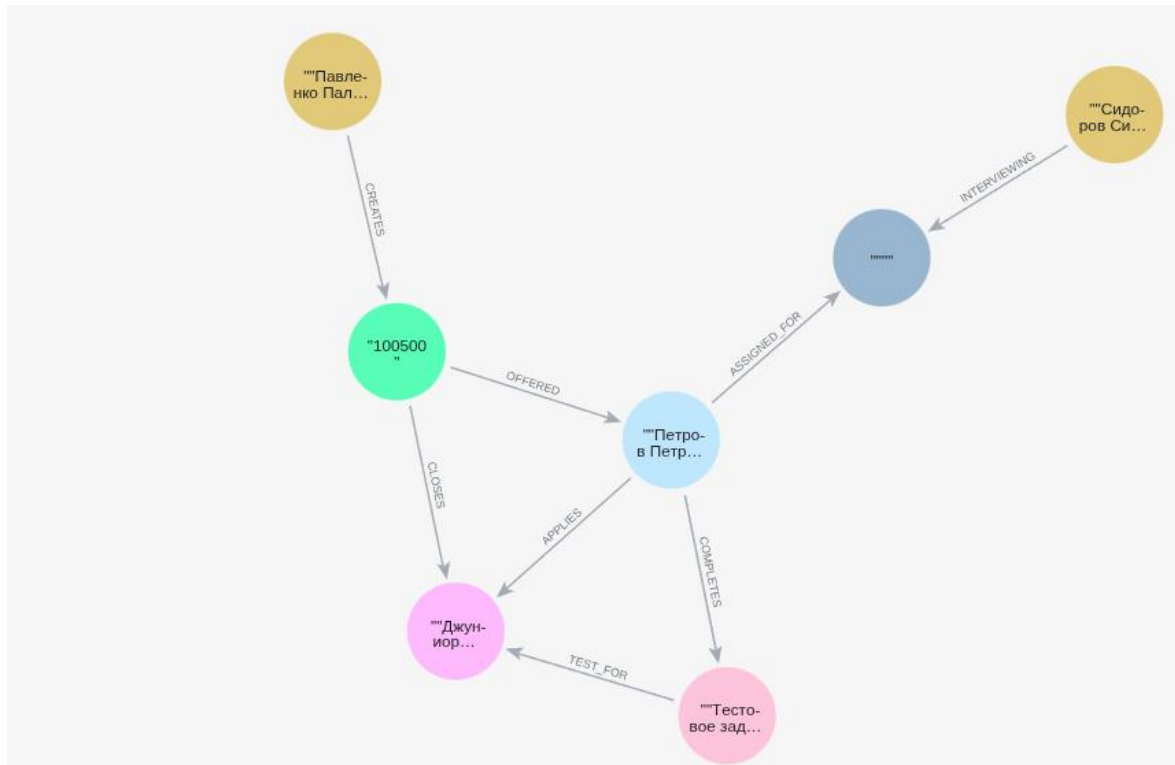


Рисунок 4 – пример заполнения БД

Примеры запросов

UC-01: Авторизация пользователя:

```
MATCH (u:User {email: $email, password_hash: $password_hash})
RETURN elementId(u) AS user_id,
       u.full_name AS full_name,
       [l IN labels(u) WHERE l IN
['Admin', 'HR', 'Tech_spec', 'Manager']][0] AS role
```

Количество запросов: 1 запрос. Не зависит от общего количества пользователей в базе.

UC-02: Создание вакансии:

```
CREATE (v:Vacancy:Open {
  title: $title,
  description: $description,
  created_at: date()
})
```

```
RETURN elementId(v) AS vacancy_id
```

Количество запросов: 1 запрос.

UC-03: Ручное добавление кандидата:

```
MERGE (c:Candidate {email: $email})
ON CREATE SET
    c:New,
    c.full_name = $full_name,
    c.phone = $phone,
    c.resume_url = $resume_url
ON MATCH SET
    c.full_name = $full_name
WITH c
MATCH (v:Vacancy) WHERE id(v) = $vacancy_id
MERGE (c)-[:APPLIES]->(v)
RETURN elementId(c) AS candidate_id
```

Количество запросов: 1 составной запрос в рамках одной транзакции.

UC-04: Редактирование информации о кандидате:

```
MATCH (c:Candidate)
WHERE id(c) = $candidate_id
SET c.full_name = $full_name,
    c.email = $email,
    c.phone = $phone
RETURN c
```

Количество запросов: 1 запрос.

UC-12 Просмотр статистики:

Статистика по статусам кандидатов для вакансии

```
MATCH (c:Candidate)-[:APPLIES]->(v:Vacancy {title: $vacancy_title})
WITH c, [l IN labels(c) WHERE l IN
['New', 'Test', 'Interview', 'Offer', 'Rejected', 'Hired']][0] AS status
RETURN status, count(*) AS count
ORDER BY count DESC
```

Количество запросов: 1 запрос.

2.2. Реляционная модель

Графическое представление модели

Структуру реляционной базы данных демонстрирует ER-диаграмма, показанная на рис. 5.

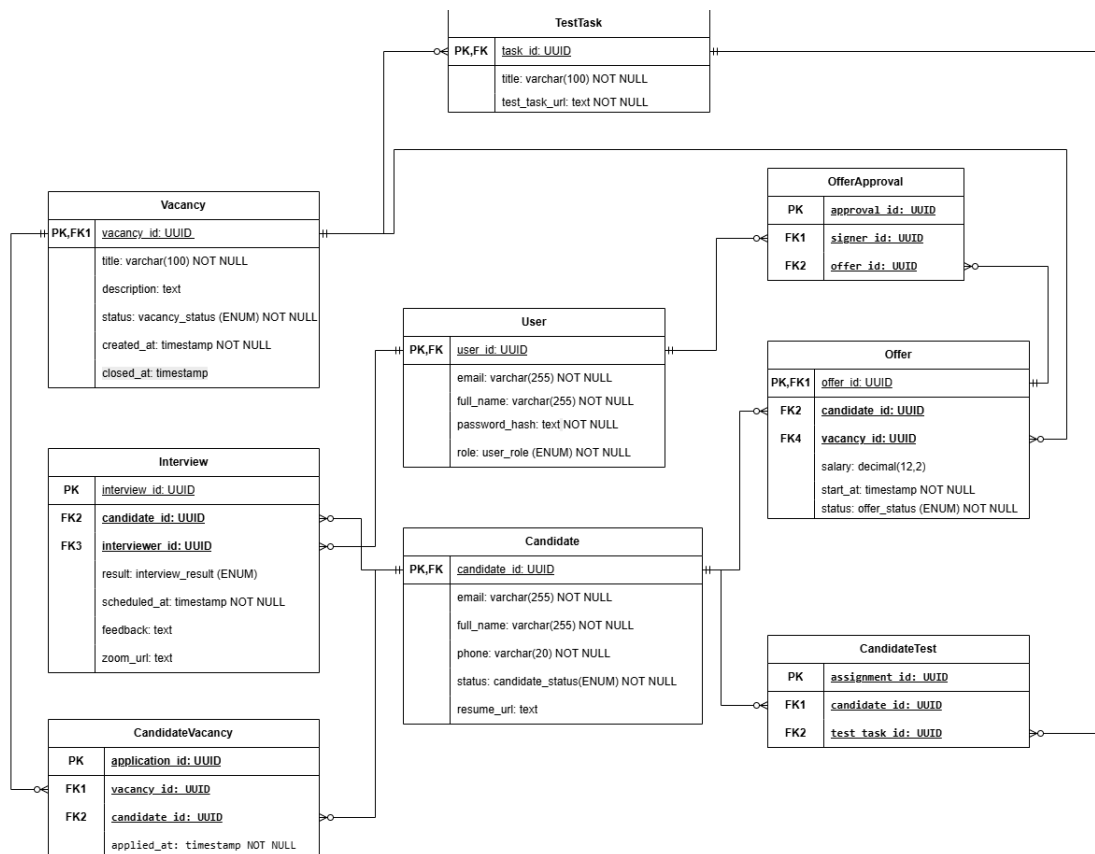


Рисунок 5 – ER-диаграмма реляционной модели

Описание сущностей и связей

1. Vacansy — Вакансия

Атрибуты Vacansy показаны в табл. 9.

Таблица 9 – атрибуты сущности Vacansy (SQL)

Атрибут	Назначение	Тип	Вес (байт)
vacancy_id	PK	UUID	16
creator_id	FK → User	UUID	16

title	Название позиции	varchar(100)	переменный: 1 или 4 + 30 = ~34
description	Описание вакансии	text	4 + 200 = ~204
status	Статус вакансии - открыта/закрыта	ENUM	4
created_at	Дата создания - для статистики (например, среднее время закрытия вакансии) и документов	timestamp	8
closed_at	Дата закрытия (NULL) - для статистики и документов	timestamp	8, если значение не NULL

Значения status: OPEN, CLOSED

Скрытые затраты:

- NULL-битовая карта: 1 байт (8 колонок)
- Заголовок: 23 байта

Итого на строку: ≈ 315 байт

2. TestTask — Тестовое задание

Тестовое задание, прикрепляется к вакансии. Может быть отправлен кандидату по кнопке "Отправить тестовое". Атрибуты TestTask показаны в табл. 10.

Таблица 10 – атрибуты сущности TestTask (SQL).

Атрибут	Назначение	Тип	Вес (байт)
task_id	PK	UUID	16
title	Название задания - для легкого опознавания задания	varchar(100)	переменный: 1 или 4 + 30

	без открытия ссылки		
test_task_url	Условия задания	text	переменный: 1 или 4 + 100

Скрытые затраты:

- Заголовок строки: 23 байта
- Выравнивание: до 8 байт

Итого на строку: ≈ 185 байт

3. User — Сотрудник системы

Пользователь CRM - HR, Технический специалист, руководитель или админ. Права доступа зависят от роли пользователя. Атрибуты сущности показаны в табл. 11.

Таблица 11 – атрибуты сущности User (SQL)

Атрибут	Назначение	Тип	Вес (байт)
user_id	PK	UUID	16
email	Логин - для входа в систему	varchar(255)	переменный: 1 или 4 + 30
full_name	ФИО	varchar(255)	переменный: 1 или 4 + 40
password_hash	Хеш пароля - для входа в систему	text	переменный: 1 или 4 + 64
role	Роль (enum) - определяет права доступа	ENUM	4

Значения role: ADMIN, HR, TECH_SPEC, MANAGER

Скрытые затраты:

- NULL-битовая карта: не нужна (все NOT NULL)
- Заголовок: 23 байта

Итого на строку: ≈ 189 байт

4. Interview — Интервью

Информация о назначенных и прошедших собеседованиях - время, результат, отзыв. С каждым интервью связан кандидат и специалист, которые должны быть на собеседовании. Атрибуты сущности показаны в табл. 12.

Таблица 12 – атрибуты сущности Interview (SQL)

Атрибут	Назначение	Тип	Вес (байт)
interview_id	PK	UUID	16
candidate_id	FK → Candidate	UUID	16
interviewer_id	FK → User	UUID	16
result	Итог (enum)	ENUM	4
scheduled_at	Дата+время - для планирования времени кандидатом и специалистом	timestamp	8
feedback	Отзыв от интервьюера о кандидате и собеседовании. Предназначен, чтобы учитывать в принятии решения о найме или для отправки обратной связи кандидату	text	переменный: 1 или 4 + 100
zoom_url	Ссылка на онлайн-встречу (NULL)	text	переменный: 1 или 4 + 50

Значение result: Awaiting Interview, Interview Passed, Interview Failed

Скрытые затраты:

- NULL-битовая карта: 1 байт
- Заголовок: 23 байта

Итого на строку: ≈ 242 байта

5. Candidate — Соискатель

Хранит данные (в том числе контактные) кандидатов. Независимая сущность, которая может существовать без вакансии. Атрибуты сущности показаны в табл. 13.

Таблица 13 – атрибуты сущности Candidate (SQL)

Атрибут	Назначение	Тип	Вес (байт)
candidate_id	PK	UUID	16
email	Почта	varchar(255)	переменный: 1 или 4 + 30
full_name	ФИО	varchar(255)	переменный: 1 или 4 + 40
phone	Телефон	varchar(20)	переменный: 1 или 4 + 20
status	Статус (enum) - определяет, на каком этапе трудоустройства находится кандидат	ENUM	4
resume_url	Ссылка на резюме (NULL) - для оценки кандидата на соответствие требованиям вакансии	text	1 или 4 + 100

Значения status: NEW, TEST, INTERVIEW, OFFER, REJECTED, HIRED

Скрытые затраты:

- Заголовок: 23 байта

Итого на строку: ≈ 249 байт

6. Offer — Оффер

Документ с финальными условиями работы. Направляется на подпись руководителю внутри CRM и кандидату. Атрибуты сущности показаны в табл. 14.

Таблица 14 – атрибуты сущности Offer (SQL)

Атрибут	Назначение	Тип	Вес (байт)
offer_id	PK	UUID	16
candidate_id	FK → Candidate	UUID	16
signer_id	FK → User	UUID	16
vacancy_id	FK → Vacancy	UUID	16
salary	Зарплата	decimal(12,2)	9
start_at	Дата выхода на работу	timestamp	8
status	Статус оффера	ENUM	4

Значения status: PENDING, APPROVED_MNG, REJECTED_MNG, APPROVED_CND, REJECTED_CNF

Скрытые затраты:

- Заголовок: 23 байта

Итого на строку: ≈ 108 байт (без текстов)

7. CandidateVacancy — Отклик

Реализация связи «многие-ко-многим» (\$M:N\$) между кандидатами и вакансиями. Эта таблица является точкой входа в воронку найма и показана на табл. 15.

Таблица 15 – атрибуты связи CandidateVacancy

Атрибут	Назначение	Тип	Вес (байт)
application_id	PK	UUID	16
vacancy_id	FK → Vacancy	UUID	16
candidate_id	FK → Candidate	UUID	16
applied_at	Время "отклика" - привязки кандидата к конкретной вакансии	timestamp	8

Скрытые затраты:

- Заголовок: 23 байта

Итого на строку: ≈ 56 байт

8. OfferApproval — Согласование оффера

Связующая таблица между оффером и руководителем(подписателем).

Создана для реализации связи «многие ко многим» и показана в табл. 16.

Таблица 16 – атрибуты связи OfferApproval

Атрибут	Назначение	Тип	Вес (байт)
approval_id	PK	UUID	16
signer_id	FK → User Кто согласовал	UUID	16
offer_id	FK → Offer Какой оффер	UUID	16

Скрытые затраты:

- Заголовок: 23 байта

Итого на строку: ≈ 71 байт

9. CandidateTest — Назначенные тесты

Связующая таблица между кандидатом и тестовым заданием и показана в табл. 17. Создана для реализации связи «многие ко многим» и назначения тестового конкретному кандидату.

Таблица 17 – атрибуты связи CandidateTest

Атрибут	Назначение	Тип	Вес (байт)
assignment_id	PK	UUID	16
candidate_id	FK → Candidate Кто выполняет	UUID	16
test_task_id	FK → TestTask Какое задание	UUID	16

Скрытые затраты:

- Заголовок: 23 байта

Итого на строку: ≈ 71 байт

Итоговое резюме по весу SQL-модели показано в табл. 18.

Таблица 18 – сводка по весу SQL-модели

Сущность	Средний вес записи
TestTask	~185 байт
Vacancy	~315 байт
User	~189 байт
Interview	~242 байта

Candidate	~249 байт
Offer	~108 байт
CandidateVacancy	~56 байт
OfferApproval	~71 байт
CandidateTest	~71 байт

Оценка объёма информации

Для сохранения в БД по одному экземпляру каждой сущности, согласно таблице выше, требуется ~1300 байт.

Зависимость объёма от числа кандидатов

Методология:

- Пусть N - число кандидатов, хранимое в базе.
- Пусть число сотрудников системы: $N/10$
- Пусть число вакансий равно числу кандидатов;
- Каждый кандидат подаёт в среднем 5 откликов;
- Каждому отклику соответствует одно собеседование;
- Каждой вакансии соответствует одно тестовое задание и один оффер.

Тогда примерный объём данных S в зависимости от числа кандидатов N выражается формулой: $S(N) \sim N \times (249 + 315 + 5 \times 56 + 5 \times 242 + 185 + 108 + 6 \times 71) + N/10 = 2800 \times N + N/10$ байт

Избыточность данных

Посчитаем полезный объем каждой сущности. В него не входит вес ключей, заголовков, оверхеда на созданные поля, NULL-битовых карт и т.п. Также пусть для одной enum величины достаточно 1 байта.

Vacancy: $30 + 200 + 8 + 8 + 1 = 247$ байт

TestTask: $30 + 100 = 130$ байт

User: $30 + 40 + 64 + 1 = 135$ байт

Interview: $8 + 100 + 50 + 1 = 159$ байт

Candidate: $30 + 40 + 20 + 100 + = 191$ байт

Offer: $9 + 8 + 1 = 18$ байт.

Рассчитанная избыточность сущностей показана в табл. 19.

Таблица 19 – избыточность сущностей в SQL-модели

Сущность	Избыточность
Vacancy	$315/247 = 1.28$
TestTask	$185/130 = 1.42$
User	$189/135 = 1.4$
Interview	$242/159 = 1.52$
Candidate	$249/191 = 1.30$
Offer	$108/18 = 6$

Средняя избыточность на узел: 2.15

Также для того, чтобы посчитать избыточность, нам надо учитывать размер промежуточных таблиц, реализующих связи "многие ко многим".

Общая избыточность:

$$\sum sql = 315 + 185 + 189 + 242 + 249 + 108 + 56 + 71 + 71$$

$$\sum clean = 247 + 130 + 135 + 159 + 191 + 18$$

$$R_coeff = \sum sql / \sum clean = 1.68$$

Таким образом, избыточность (Redundancy) равна $R \approx 1.68 \times V$, где V - объем полезных данных. Или $R \approx 2.15 \times N$, где N - количество узлов.

Направление роста

Между размером БД и количеством записей в таблицах прямая зависимость. Это применимо ко всем сущностям.

Добавление n экземпляров сущности увеличивает размер на $n * \text{size}$, где size равен среднему весу записи этой сущности.

На вес также влияет количество связей между сущностями. В реляционной БД связи реализуются через промежуточные таблицы. Создание каждой записи в такой таблице увеличивает вес БД.

- Отклик кандидата на вакансию (CandidateVacancy) добавляет 56 байт.
- Выдача тестового кандидату (CandidateTest) добавляет 71 байт.
- Назначение интервью (Interview) — это отдельная сущность, которая содержит внешние ключи на Candidate и User. Вес одного интервью — 242 байта.
- Создание оффера (Offer) добавляет 108 байт (сама запись). Также с каждым оффером также создаётся запись в таблице OfferApproval (+71 байт).

Таким образом, наибольший рост база будет испытывать при создании новых офферов ($108 + 71 = 179$ байт) и при назначении интервью (242 байта).

Примеры данных и запросов

1. Сотрудники (Users)

```
INSERT INTO "User" (email, full_name, password_hash, role) VALUES
('admin@it-corp.ru', 'Екатерина Максимова', 'hash_admin', 'admin'),
('recruiter_julia@it-corp.ru', 'Юлия Сергеева', 'hash_hr', 'hr'),
('tech_lead_dima@it-corp.ru', 'Дмитрий Белов', 'hash_lead',
'tech_spec'),
('cto@it-corp.ru', 'Артем Николаев', 'hash_cto', 'manager');
```

Результат выполнения запроса показан в табл. 20.

Таблица 20 – результат добавления Users

user_id	email	full_name	password_hash	role
f52251a9-ea39-460a-8f6c-a939c79e0535	admin@it-corp.ru	Екатерина Максимова	hash_admin	admin
2a88bc69-db69-4df8-a1fd-deeacbfe1d0b	recruiter_julia@it-corp.ru	Юлия Сергеева	hash_hr	hr
643a9ad3-8f43-4329-bcee-d1bfe88299bf	tech_lead_dima@it-corp.ru	Дмитрий Белов	hash_lead	tech_spec
ff00d866-b6f7-4f25-acd6-73e041b3d695	cto@it-corp.ru	Артем Николаев	hash_cto	manager

2. Вакансии (Vacancy)

```

INSERT INTO Vacancy ( title, description, status, created_at)
SELECT
    'System Analyst',
    'Проектирование сложных архитектурных решений и интеграций.',
    'open',
    NOW() - INTERVAL '7 days'
FROM "User" u
WHERE u.email = 'admin@it-corp.ru';

INSERT INTO Vacancy (title, description, status)
VALUES ('Название вакансии', 'Описание до 5000 символов', 'open')
RETURNING vacancy_id;

```

Результат добавления вакансий показан в табл. 21.

Таблица 21 – результат добавления Vacancy

vacancy_id	title	description	status	created_at	closed_at
5518fc82-ec74-4c69-b308-	System Analyst	Проектирование сложных архитектур	open	2026-03-28 17:15:59.401	

6282a6efe5ba		ых решений и интеграций.		134	
5a2cce69-6371-4dcb-aa4c-4fe8b1869f4f	Название вакансии	Описание до 5000 символов	open	2026-04-04 17:15:59.409 259	

3. Кандидаты (Candidate)

```
INSERT INTO Candidate (email, full_name, phone, status, resume_url)
VALUES
    ('ivanov@mail.ru', 'Иван Иванов', '+79991234567', 'interview',
    'http://s3.storage/resume1.pdf'),
    ('petrova@yandex.ru', 'Анна Петрова', '+79997654321', 'test',
    'http://s3.storage/resume2.pdf');
```

Результат выполнения запроса показан в табл. 23.

Таблица 22 – результат добавления Candidate

candidate_id	email	full_name	phone	status	resume_url
26e36071-0286-4b9f-9f02-8558bffcde77	ivanov@mail.ru	Иван Иванов	+79991234567	interview	http://s3.storage/resume1.pdf
bd0378c0-b90a-487f-9153-c569136f4c21	petrova@yandex.ru	Анна Петрова	+79997654321	test	http://s3.storage/resume2.pdf

4. Тестовые задания (TestTask)

```
INSERT INTO TestTask (vacancy_id, title, test_task_url)
SELECT
    v.vacancy_id,
    'Архитектурный кейс по интеграциям',
    'http://company.docs/tasks/sa_case'
```



```
FROM Vacancy v
WHERE v.title = 'System Analyst';
```

Результат выполнения запроса показан в табл. 23.

Таблица 23 – результат добавления TestTask

task_id	vacancy_id	title	test_task_url
71541210-2c2e-4af2- aee0-aa68509bcef9	f4074346-bc5d-4bd7- b3c0-2c4cd7d98edb	Архитектурный кейс по интеграциям	http://company.docs/t asks/sa_case

5. Отклик кандидата (CandidateVacancy)

```
INSERT INTO CandidateVacancy (vacancy_id, candidate_id)
SELECT
    v.vacancy_id,
    c.candidate_id
FROM Vacancy v
JOIN Candidate c ON c.email = 'ivanov@mail.ru'
WHERE v.title = 'System Analyst';
```

```
INSERT INTO CandidateVacancy (vacancy_id, candidate_id)
SELECT
    v.vacancy_id,
    c.candidate_id
FROM Vacancy v
JOIN Candidate c ON c.email = 'petrova@yandex.ru'
WHERE v.title = 'System Analyst';
```

Результат выполнения запроса показан в табл. 24.

Таблица 24 – добавление отклика кандидата

application_id	vacancy_id	candidate_id	applied_at
20bcd7a8-329c-41f3- ae8f-a779e47427e1	18cf1baf-e37b-4154- 8a71-48aff0854061	eb177d65-be13-44f2- bd92-c0ad66bca2f0	2026-04-04 17:18:15.444977
35e86fff-3be7-4906- 981f-81bf0e635826	18cf1baf-e37b-4154- 8a71-48aff0854061	1bf44069-c67d-4cb0- b0dd-d704e4391d83	2026-04-04 17:18:15.446214

6. Прохождение тестового (CandidateTest)

```
INSERT INTO CandidateTest (candidate_id, test_task_id)
SELECT
    c.candidate_id,
    t.task_id
FROM Candidate c
JOIN TestTask t ON t.title = 'Архитектурный кейс по интеграциям'
WHERE c.email = 'ivanov@mail.ru';
```

Результат выполнения запроса показан в табл. 25.

Таблица 25 – добавление тестового задания

assignment_id	candidate_id	test_task_id
ed9b623d-cb7f-4371-ac5f-2a2d76455e76	08bba9ae-161b-4838-bf75-24f9ab7b7afc	a3f31e22-4471-47ca-bedd-8dde3d4e4237

7. Интервью (Interview)

```
INSERT INTO Interview (candidate_id, interviewer_id, result,
scheduled_at, feedback, zoom_url)
SELECT
    c.candidate_id,
    u.user_id,
    'interview_passed',
    NOW() - INTERVAL '2 days',
    'Глубокое понимание REST API и UML.',
    'https://zoom.us/j/interview1'
FROM Candidate c
JOIN "User" u ON u.email = 'tech_lead_dima@it-corp.ru'
WHERE c.email = 'ivanov@mail.ru';
```

Результат выполнения запроса показан в табл. 26.

Таблица 26 – добавление интервью

interview_id	candidate_id	interviewer_id	result	scheduled_at	feedback	zoom_url
b85dff51-569e-4e1a-aec4-c2adcaca9113	695f5c62-e04f-4d3f-a5b3-e5a3d3bc1c23	14eea328-53d7-42b6-bf67-0b1584eace5	interview_passed	2026-04-02 17:20:01.74417	Глубокое понимание REST API и UML.	https://zoom.us/j/interview1

8. Оффер (Offer)

```
INSERT INTO Offer (candidate_id, vacancy_id, salary, start_at,
status)
```

```
SELECT
    c.candidate_id,
    v.vacancy_id,
    200000.00,
    NOW() + INTERVAL '14 days',
    'approved_mng'
FROM Candidate c
JOIN Vacancy v ON v.title = 'System Analyst'
JOIN "User" u ON u.email = 'cto@it-corp.ru'
WHERE c.email = 'ivanov@mail.ru';
```

Результат выполнения запроса показан в табл. 27.

Таблица 27 – добавление оффера

offer_id	candidate_id	vacancy_id	salary	start_at	status
80ff52b9-b814-4a57-8342-e378db72de8c	83ea07cc-a072-4ba8-9789-eb4bc5fddeba	001693bc-2868-4945-9f87-5e8de7d6f601	200000.00	2026-04-18 17:22:22.062242	approved_mng

9. Согласование оффера (OfferApproval)

```
INSERT INTO OfferApproval (signer_id, offer_id)
```

```
SELECT
```

```

        u.user_id,
        o.offer_id
FROM "User" u
JOIN Candidate c ON c.email = 'ivanov@mail.ru'
JOIN Offer o ON o.candidate_id = c.candidate_id
WHERE u.email = 'cto@it-corp.ru';

```

Результат выполнения запроса показан в табл. 28.

Таблица 28 – согласование оффера

approval_id	signer_id	offer_id
7935f1b1-710b-4fec-82a2-21d723202aeb	8f71366b-0de4-452d-908f-be47fe99c734	0ea1098e-841e-4c25-87ac-c30ef6ab516b

Примеры запросов

UC-01 Авторизация пользователя

Проверка учетных данных и получение роли

```

SELECT user_id, full_name, role
FROM "User"
WHERE email = 'admin@it-corp.ru'
      AND password_hash = 'hash_admin';

```

Количество запросов: 1 запрос. Не зависит от общего количества пользователей в базе (при наличии уникального индекса/B-Tree по полю email).

Задействованные таблицы: 1 ("User").

UC-02 Создание вакансии

```

INSERT INTO Vacancy (title, description, status)
VALUES ('Название вакансии', 'Описание до 5000 символов', 'open')
RETURNING vacancy_id;

```

Количество запросов: 1 запрос. Время выполнения константно и не зависит от текущего объема вакансий в базе.

Задействованные таблицы: 1 (Vacancy).

UC-03 Ручное добавление кандидата

```

BEGIN;

WITH new_candidate AS (
    INSERT INTO Candidate (full_name, email, phone, resume_url,
status)
    VALUES (
        'Иванов Иван Иванович', -- NOT NULL
        'ivan@example.com',      -- NOT NULL, UNIQUE
        '+79991234567',          -- NOT NULL
        'http://resume.url',
        'new'                    -- NOT NULL (Статус из твоего
сценария)
    )
    -- Если кандидат уже есть, мы не создаем дубль, а берем его ID.
    -- Это решает проблему "Кандидат уже в базе, но хочет на другую
вакансию".
    ON CONFLICT (email) DO UPDATE
    SET full_name = EXCLUDED.full_name
    RETURNING candidate_id
)
INSERT INTO CandidateVacancy (candidate_id, vacancy_id)
SELECT nc.candidate_id, v.vacancy_id
FROM new_candidate nc
CROSS JOIN Vacancy v
WHERE v.vacancy_id = 'b919a163-9417-4954-ae78-39e496f5d261'; --
Проверка существования вакансии

COMMIT;

```

Количество запросов: 1 составной запрос в рамках одной транзакции. Благодаря использованию CTE (Common Table Expressions) и конструкции ON CONFLICT, приложение делает всего один вызов к БД. Это решает потенциальную проблему 2-х запросов (проверка существования -> создание). Но внутренних операций (что БД сделала под капотом) 3 - INSERT/UPDATE (Candidate) -> SELECT (Vacancy) -> INSERT (Link). Зависимости от количества кандидатов в базе нет.

Задействованные таблицы: 3 (Candidate — запись/обновление, CandidateVacancy — запись, Vacancy — чтение/проверка наличия).

УС-04 Редактирование информации о кандидате

```
UPDATE Candidate
SET
    full_name = 'Иванов Иван',
    email = 'newemail@example.com',
    phone = '+79001112233'
WHERE candidate_id = 'uuid_кандидата'
RETURNING *;
```

Количество запросов: 1 запрос. Не зависит от объема БД (обновление происходит по Primary Key candidate_id, что обеспечивает мгновенный доступ).

Задействованные таблицы: 1 (Candidate).

УС-12 Просмотр статистики

Статистика по статусам кандидатов для вакансии

```
SELECT c.status, COUNT(*) as count
FROM Candidate c
JOIN CandidateVacancy cv ON cv.candidate_id = c.candidate_id
JOIN Vacancy v ON v.vacancy_id = cv.vacancy_id
WHERE v.title = 'System Analyst' -- фильтр по вакансии
GROUP BY c.status
ORDER BY count DESC;
```

Количество запросов: 1 запрос. Приложение делает один запрос, однако сложность его выполнения внутри СУБД зависит от количества кандидатов, привязанных к конкретной вакансии (числа записей в связующей таблице CandidateVacancy для искомого vacancy_id) - $O(K)$, где K — это количество откликов на конкретную вакансию.

Задействованные таблицы: 3 (Candidate, CandidateVacancy, Vacancy).

УС-13 Полный экспорт (Backup)

```
-- Экспорт всех данных в JSON (через агрегацию)
```

```

SELECT jsonb_build_object(
    'users', (SELECT jsonb_agg(to_jsonb("User")) FROM "User"),
    'vacancies', (SELECT jsonb_agg(to_jsonb(Vacancy)) FROM
Vacancy),
    'candidates', (SELECT jsonb_agg(to_jsonb(Candidate)) FROM
Candidate),
    'candidate_vacancies', (SELECT
jsonb_agg(to_jsonb(CandidateVacancy)) FROM CandidateVacancy),
    'test_tasks', (SELECT jsonb_agg(to_jsonb(TestTask)) FROM
TestTask),
    'candidate_tests', (SELECT jsonb_agg(to_jsonb(CandidateTest))
FROM CandidateTest),
    'interviews', (SELECT jsonb_agg(to_jsonb(Interview)) FROM
Interview),
    'offers', (SELECT jsonb_agg(to_jsonb(Offer)) FROM Offer),
    'offer_approvals', (SELECT jsonb_agg(to_jsonb(OfferApproval))
FROM OfferApproval)
) AS full_backup;

```

Количество запросов: 1 запрос. Это крайне тяжелый аналитический запрос. Хотя сетевой вызов всего один, база данных будет выполнять полное сканирование (Seq Scan) указанных таблиц. Время выполнения и потребление памяти напрямую зависят от суммарного количества объектов во всей БД.

Задействованные таблицы: 9 ("User", Vacancy, Candidate, CandidateVacancy, TestTask, CandidateTest, Interview, Offer, OfferApproval).

2.3. Сравнение моделей

UC-01 Авторизация пользователя

- SQL: 1 запрос. Задействованные таблицы: 1 ("User").
- NoSQL: 1 запрос.

Модели равны

UC-02 Создание вакансии

- SQL: 1 запрос. Задействованные таблицы: 1 (Vacancy).
- NoSQL: 1 запрос.

Модели равны

UC-03 Ручное добавление кандидата

- SQL: 1 составной запрос (3 подзапроса) внутри одной транзакции
- NoSQL: 1 составной запрос (2 подзапроса) внутри одной

транзакции

NoSQL короче и быстрее при глубоких связях засчет графовой структуры и прохода по рёбрам

UC-04 Редактирование информации о кандидате

- SQL: 1 запрос. Задействованные таблицы: 3 (Candidate — запись/обновление, CandidateVacancy — запись, Vacancy — чтение/проверка наличия).
- NoSQL: 1 запрос. Задействованные таблицы: 3 (Candidate — запись/обновление, CandidateVacancy — запись, Vacancy — чтение/проверка наличия).

Модели равны

UC-12 Просмотр статистики

- SQL: 1 запрос (несколько JOIN). Задействованные таблицы: 3 (Candidate, CandidateVacancy, Vacancy).
- NoSQL: 1 запрос.

NoSQL быстрее засчет графовой структуры и прохода по рёбрам

UC-13 Полный экспорт (Backup)

- SQL: 1 тяжелый составной запрос внутри одной транзакции. Задействованные таблицы: 9 ("User", Vacancy, Candidate, CandidateVacancy, TestTask, CandidateTest, Interview, Offer, OfferApproval).
- NoSQL: 1 тяжелый составной запрос внутри одной транзакции.

Оба запроса достаточно трудозатратные при большом объеме данных.
Табличная структура лучше для создания дампов.

Итог по объёму хранимых данных:

Зависимость объёма хранимых данных от числа кандидатов N :

- SQL: $2800 * N + N/10$ байт
- NoSQL: $3500N + N/10$ байт Избыточность:
- SQL: $R \approx 1.68 * V$, где V - объем полезных данных. Или $R \approx 2.15 *$

N , где N - количество узлов.

- NoSQL: $R \approx 2.03 * V$, где V - объем полезных данных. Или $R \approx 2.95 * N$, где N - количество узлов.

По объёму хранимых данных при прочих равных SQL выгоднее neo4j. В основном, это объясняется большими скрытыми затратами neo4j хранение свойств.

Итог по запросам:

- Благодаря графовой структуре на большинстве запросов выигрывает neo4j.
- Это достигается за счёт графовой структуры и возможностью простого и быстрого прохода по рёбрам.
- Для SQL в то же время требуются JOIN и подзапросы.
- Также запросы в neo4j проще и немного более лаконичные за счёт графовых свойств данных.

3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

3.1. Краткое описание

Приложение реализовано как классическое web-приложение и состоит из 3 частей: СУБД, бэкенд, фронтенд. Каждая из 3 частей развёртывается в отдельном docker-контейнере, организация которых происходит с помощью docker compose. Развёртывание приложения, инициализация данными происходит автоматически.

1. База данных

В качестве базы данных выступает СУБД neo4j. Доступ к ней возможен только через бэкенд посредством предоставляемого python-драйвера. Для проверки работоспособности базы данных реализован healthcheck.

2. Бэкенд

Бэкенд реализован с помощью FastAPI и выполнен по классической схеме: уровень роутеров для предоставления эндпоинтов HTTP API, уровень сервисов для реализации бизнес-логики и уровень репозитория для взаимодействия с базой данных и реализации запросов к базе данных. Для проверки работоспособности бэкенда написаны множественные интеграционные и API тесты.

3. Фронтенд.

Фронтенд веб-приложения написан с использованием React и TypeScript. Сборка фронтенда осуществляется с помощью Vite. Взаимодействие фронтенда и бэкенда реализовано в соответствии с API (файл api.ts).

Приложение состоит из компонент: основной компонент App.tsx реализует роутинг (маршрутизацию). Также реализованы компоненты, описывающие основные страницы приложения (страницы со списками сущностей, страницы

конкретного объекта). Также имеются компоненты, описывающие формы для создания и редактирования объектов, для их фильтрации, для их отображения в табличном виде. Переиспользование: компоненты таблицы и фильтров используются на всех страницах со списками сущностей. (В конструкторы компонент передаются соответствующие пропсы).

Также используются кастомные хуки: для реализации прав доступа на фронтенде, а также для установки и обновления фильтров.

3.2. Используемые технологии

Фронтенд написан на React и TypeScript, Vite для сборки. Бэкенд написан на Python, используется FastAPI. Для просмотра API используется Swagger. Взаимодействие фронтенда и бэкенда реализовано с помощью http запросов, реализованных по принципам ResttAPI. Тесты бэкенда реализованы с помощью PyTest. Графовая база данных — Neo4j, бэкенд взаимодействует с базой через стандартный драйвер. Для развертывания приложения используются Docker контейнеры, развёртываемые с помощью Docker-compose.

3.3. Снимки экрана приложения

Окно логина показано на рис. 6.

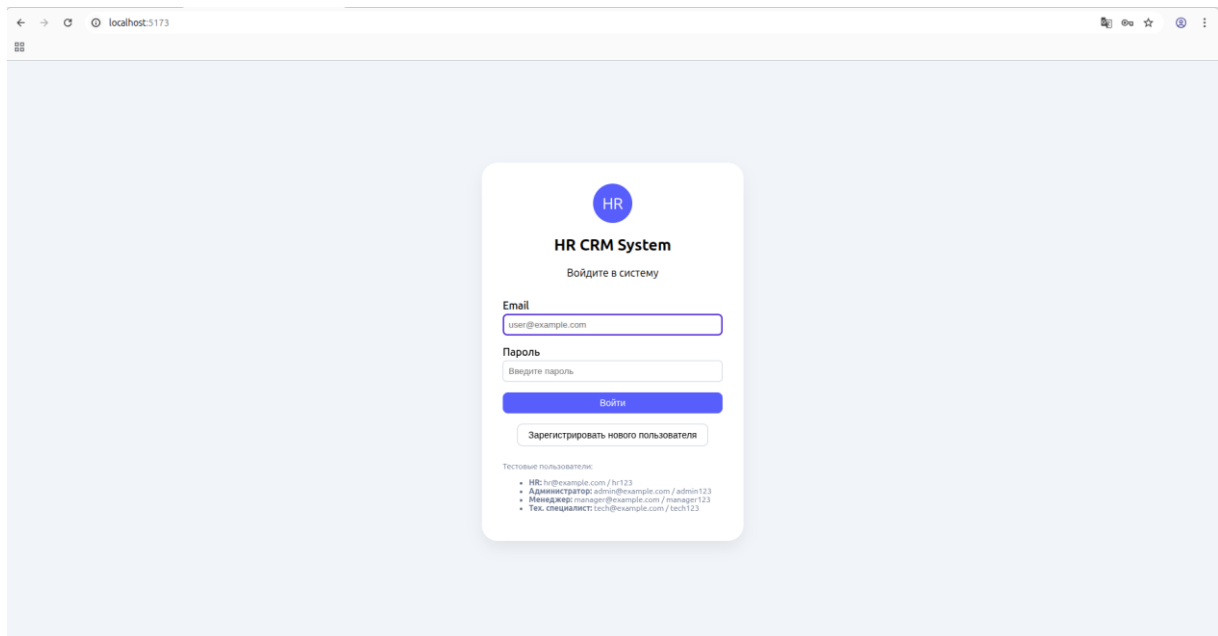


Рисунок 6 – окно логина

Страница с кандидатами показана на рис. 7.

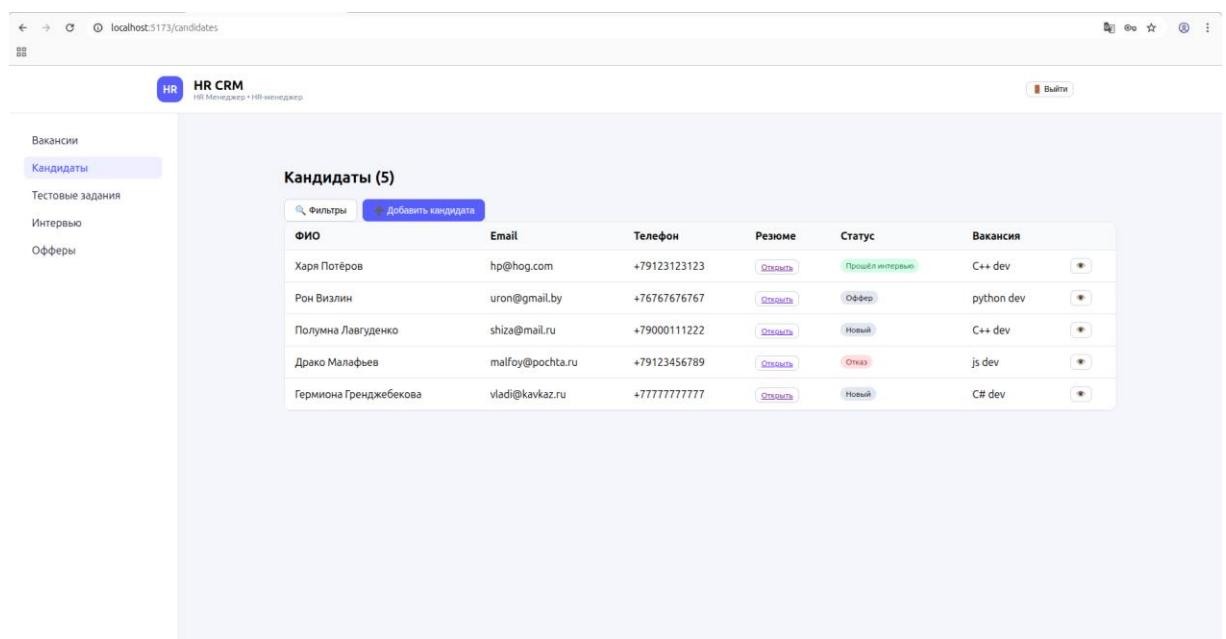


Рисунок 7 – страница с кандидатами

Страница с карточкой кандидата показана на рис. 8.

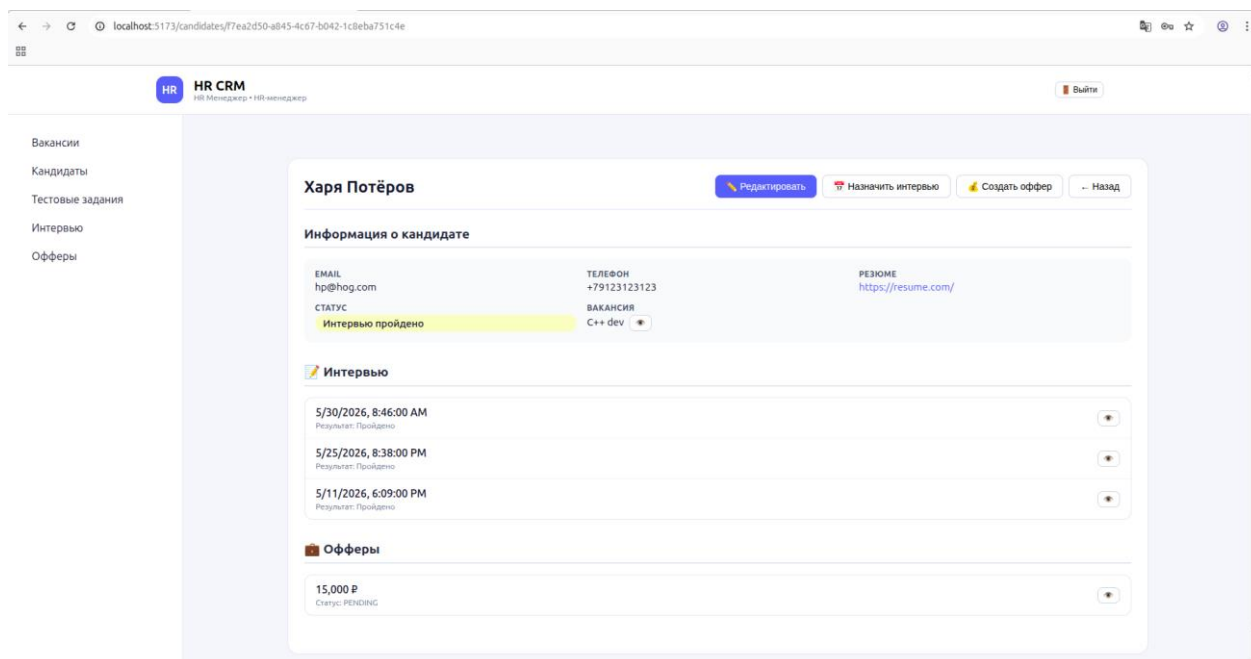


Рисунок 8 – страница кандидата

Форма добавления кандидата показана на рис. 9.

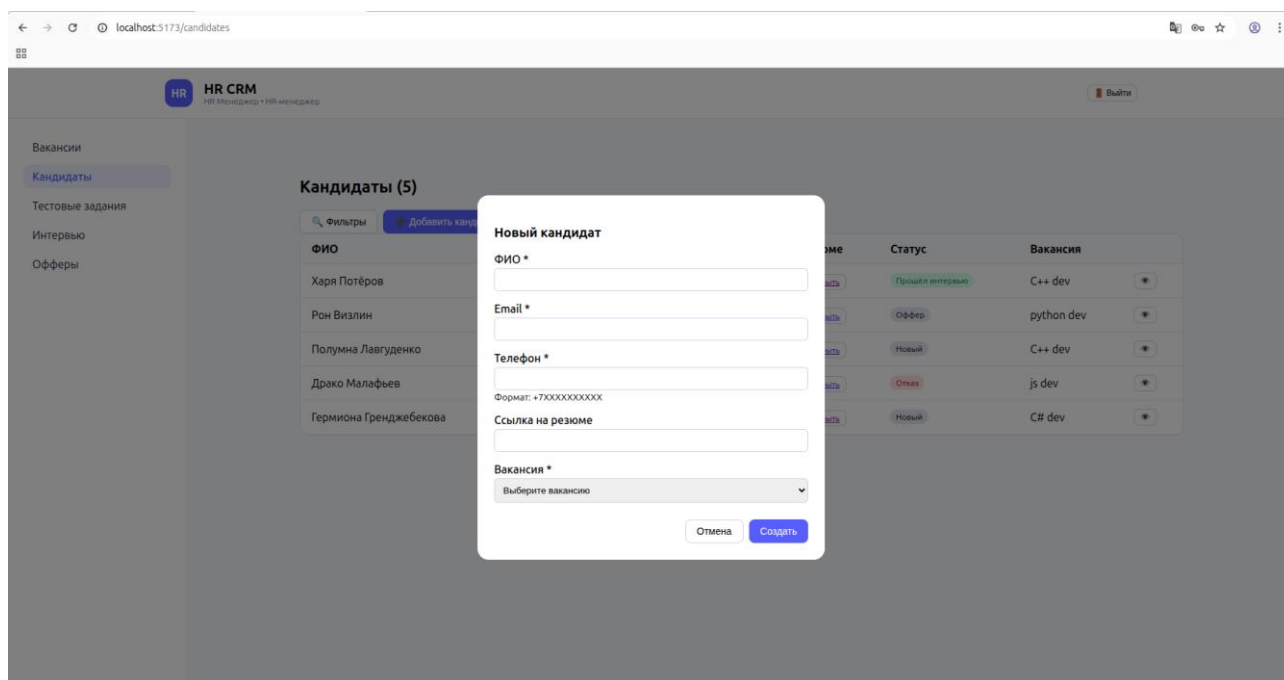


Рисунок 9 – форма добавления кандидата

Форма редактирования кандидата показана на рис. 10.

The screenshot displays the HR CRM interface. On the left is a sidebar menu with items: 'Вакансии', 'Кандидаты', 'Тестовые задания', 'Интервью', and 'Офферы'. The main content area shows the profile of 'Гермиона Гренджева'. A modal window titled 'Редактирование кандидата' is open in the center, containing the following fields:

- ФИО ***: Гермиона Гренджева
- Email ***: vladi@kavkaz.ru
- Телефон ***: +77777777777 (Format: +7XXXXXXXXXX)
- Ссылка на резюме**: https://onlyfans.com/
- Вакансия ***: CS dev (dropdown menu)
- Статус**: Новый (dropdown menu)

At the bottom of the modal are 'Отмена' and 'Сохранить' buttons. In the background, the candidate's profile card shows contact information (EMAIL: vladi@kavkaz.ru, STATUS: Новый), interview status (Нет интервью), and offer status (Нет офферов). Action buttons at the top right of the profile card include 'Назначить интервью', 'Создать offer', and 'Назад'.

Рисунок 10 – Редактирование кандидата

ЗАКЛЮЧЕНИЕ

В результате выполнения работы было разработано web-приложение, реализующее автоматизацию и отслеживание процесса найма сотрудников. Приложение позволяет:

- Создавать вакансии, регистрировать кандидатов, назначать им тестовые задания и интервью, предлагать офферы;
- Отслеживать текущий статус кандидатов, вакансий, офферов изменять их;
- Находить и анализировать кандидатов, вакансии, офферы и другие сущности с помощью обширной системы фильтрации, отслеживающей глубокие связи в приложении;
- Проводить массовый импорт или экспорт всех данных приложения в формате JSON.

Среди основных недостатков текущего решения можно выделить: отсутствие подробной аналитики данных приложения, её визуализации; не полностью протестированный программный код бэкенда и фронтенда; визуальные недостатки пользовательского интерфейса, низкая скорость работы на отдельных сценариях.

Таким образом, основные пути улучшения приложения:

- Реализация подробной статистики по данным, хранимым в приложении;
- Доработка и улучшение пользовательского интерфейса и пользовательского опыта использования;
- Покрытие фронтенда и бэкенда приложения множественными тестами;
- Анализ и оптимизация запросов к базе данных.

Развитие данного решения предполагает устранение указанных недостатков, развёртывание приложения в продакшене, расширение поддерживаемых пользовательских сценариев. После развертывания решения возможен сбор обратной связи и улучшение решения по ней.

СПИСОК ЛИТЕРАТУРЫ

1. Документация neo4j. URL: <https://neo4j.com/docs/> (дата обращения: 20.03.2026).
2. Документация React. URL: <https://ru.legacy.reactjs.org/docs/getting-started.html> (дата обращения: 18.04.2026).
3. Документация FastAPI. URL: <https://fastapi.tiangolo.com/ru/> (дата обращения: 25.03.2026).
4. Репозиторий с проектом на Github. URL: <https://github.com/moevm/nsql1h26-hr/tree/main> (дата обращения: 21.05.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А: ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЁРТЫВАНИЮ ПРИЛОЖЕНИЯ

Для сборки и развертывания приложения необходимо наличие Docker и Docker Compose. Переменные окружения задаются в файле `.env`: `SECRET_KEY`, `NEO4J_USER`, `NEO4J_PASSWORD` (`VITE_API_URL`, `BACKEND_PORT` и `FRONTEND_PORT` — при необходимости для переопределения адреса API и портов сервисов). Сборка образов для backend (на основе Python 3.12) и frontend (на основе Node 18) выполняется командой `docker compose up -d --build`. Код копируется внутрь образов на этапе сборки, поэтому локальные директории не монтируются, что обеспечивает изоляцию и безопасность в продакшене.

Контейнер backend ожидает готовности Neo4j через механизм healthcheck, контейнер frontend зависит от backend-сервиса. Точкой входа для backend-контейнера служит скрипт `entrypoint.sh`, который выполняется при запуске, он заполняет БД тестовыми пользователями. Все сервисы привязаны к `127.0.0.1`, что делает их доступными только с локальной машины — для внешнего доступа требуется обратный прокси (например, Nginx). После успешного запуска backend будет доступен по адресу `http://127.0.0.1:8000` (или настроенному `BACKEND_PORT`), frontend — на `http://127.0.0.1:5173`, а Swagger UI для просмотра спецификации `openapi.yaml` — на порту 8080. Также предусмотрен процесс управления с использованием `makefile`:

- `make up` — сборка образов, или запуск контейнеров,
- `make down` — остановка контейнеров,
- `make front-logs` — вывод логов контейнера фронтенда,
- `make back-logs` — вывод логов контейнера бэкенда,
- `make tests` — запуск тестов бэкенда,
- `make clean` — полная очистка смонтированных volumes с данными БД.

ПРИЛОЖЕНИЕ Б: ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ

Для запуска приложения необходимо клонировать репозиторий и в корне проекта выполнить в bash следующую команду: `make up`

После успешной сборки и запуска контейнеров приложение будет доступно на `localhost:5173`

Пользователя встретит окно логина. Можно войти в приложение под учётной записью одного из 4-х тестовых пользователей (HR, тех. специалист, менеджер, администратор) или зарегистрировать новый «аккаунт».

В левой боковой панели будут отображены страницы, доступные пользователю в соответствии с его ролью. На каждой странице представлены соответствующие данные в табличном виде.

Над таблицей имеются кнопки для добавления новой записи и для отображения фильтров.

Записи в таблице отображают основную информацию каждого объекта (в соответствии с моделью данных). Также имеется кнопка с глазом («»), нажав на неё можно перейти на страницу конкретного объекта.

На странице объекта также предоставлена информация о нём, имеется кнопка для редактирования. В некоторых случаях информационные поля объекта могут ссылаться на другие записи (объекты) — в таком случае будет иметься кнопка с глазом, ведущая на соответствующую страницу.

Выйти на страницу логина можно по кнопке в правом верхнем углу.

Далее целесообразно описать специфические действия для каждой роли пользователя:

1. HR

На странице конкретной вакансии доступна возможность прикрепления тестового к вакансии через соответствующую форму

На странице конкретного кандидата доступна возможность назначения интервью и назначения оффера. Также через формы.

2. Администратор

На странице с пользователями доступны инструменты массового импорта/экспорта БД.

Доступно редактирование всех сущностей.

3. Менеджер

На странице конкретного кандидата доступна возможность назначения оффера.

На странице конкретного оффера доступно одобрение и отклонение оффера. (Осуществляется в 2 шага: сначала выбор самого менеджера, затем подтверждение выбора кандидата).

4. Тех. специалист

На странице конкретного интервью доступна возможность оставить отзыв по итогам собеседования. (Пройдено или нет, а также текст комментария). Этот функционал доступен только для тех интервью, на которые данный пользователь назначен интервьюером.